DAA Project

Part #01

NAME: Muhammad Abu Bakar, Furqan, Abdullah
NUID: 24k-0826, 24k-0527, 24k-0764
SECTION: BCS-5D

Q1 Part g)

ALGORITHM :-

The best trade lies entirely in these ~~1st~~ of the 3 conditions

(a) Lies entirely in 1st half &.
(b) Lies entirely in 2nd half &.
(c) Buys in 1st half & sells in 2nd half.

for the 3rd case the best choice would be the cheapest day in the 1st half & the most expensive day of the 2nd half. We can identify this by a linear scan on the given data structure.

PSEUDOCODE :-

```
algo& BestTrade (low, high)

    if    low >= high
          then  return None

mid  ← floor ((low + high)/2)
left ← BestTrade (low, mid)
right ← BestTrade (mid+1, high)

i ← index of min price in
      p[low .... mid]
j ← index of max price in
      p[mid+1 .... high]

cross ← (p[j] - p[i], i, j)

return   the best of left/
         right/ cross
         whose profit > 0,
          else none
```

RECURRENCE RELATION:-

$$T(n) = 2T(n/2) + cn$$

with $T(1) = c'$

SOLVING THE RECURRENCE:-

$$T(n) = 4T\left(\frac{n}{4}\right) + 2cn$$

$$T(n) = 8T\left(\frac{n}{8}\right) + 3cn$$

After $\log_2 n$ expansions:-

$$T(n) = n \cdot T(1) + cn \log_2 n$$

Result:- $T(n) = O(n \log n)$

COMPLEXITY:-

Time :- $O(n \log n)$

Space:- $O(\log n)$ for the recursion stack.

CORRECTNESS :-

An optimal pair $(i < j)$ lies both in the 1st half, days both in the

2nd half, or i in the 1st & j in the 2nd.

The recursion handles the first 2 cases

For the 3rd p[j] - p[i] is the largest for the cheapest day in the 1st half & the most expensive day in the 2nd. The algo returns the best of the 3 cases.

Q1) part h

EX No1 :- MEDIAN OF 2 DATABASES ALGORITHM:-

Query the median of the current active range in both Database A & B.
Compare the 2 values [amid] & [bmid]. If [a mid] < [b mid] eliminate the bottom half of A. & the top half of B.
If [bmid] < [amid]

eliminate the bottom half of B & the top half of A. Recursively repeat this process on the remaining elements until only 1 candidate remains in each database.

PSEUDOCODE:-

algo FindOverallMedian (D1, D2, n)

return BinarySearch (D1, D2, 1, n, 1, n)

func BinarySearch (D1, D2, st1, en1, st2, en2)

length ← end1 - st1 + 1

if length == 1
then return min (D1.query(st1), D2.query(st2))

mid1 ← st1 + floor((length-1)/2)
mid2 ← st2 + floor((length-1)/2)

a_mid ← D1.query(mid1)
b_mid ← D2.query(mid2)

if a_mid < b_mid
then
return BinarySearch(D1, D2, mid1+1, en1, st2, mid2)
else
return BinarySearch(D1, D2, st1, mid1, mid2+1, en2)

RECURRENCE RELATION:-

$$Q(n) = Q\left(\frac{n}{2}\right) + 2$$

SOLVING THE RECURRENCE:-

$$Q(n) = Q(n/2) + 2$$

$$Q(n) = Q(n/4) + 4$$

pattern observed as

$$Q(n) = Q\left(\frac{n}{2^k}\right) + 2^k k$$

Base case Reaches When:-

Subset size = 1

So $\frac{n}{2^k} = 1$

$k = \log_2 n$

$O(n) = 2 \log_2(n)$

FINAL RESULT :-

$O(\log n)$

CORRECTNESS :-

let amid & bmid = Medians of the current valid ranges in D1 & D2.

(•) If [amid] < [bmid] all elements in th bottom half of D1 are less than amid, which itself is less than the top half of ~~D2~~ D2. The bottom half elements are mathematically too small to reach the nth overall

rank, hence they can be discarded. Symmetrically same logic for when [b mid] >< [a mid].

(6) Discarding these 2 halves maintain the exact same relevant rank for the true median with in the remaining pool.

EX #02 SIGNIFICANT INVERSIONS

ALGORITHM :-

Merge sort & count, but merge order doesn't reveal $a_i > 2a_j$ so the cross pairs are counted in a separate linear pass. over the 2 sorted halfs. For each y in the sorted right halfs advance a pointer p while L[p] <= 2y then add |L| - p. Finally merge normally.

PSEUDOCODE:-

```
algo   Sort & Count Sig(A)

    if |A| <= 1
        then return (A,0)

(L,cL) ← Sort & CountSig(left half)
(R,cR) ← Sort & Count Sig(Right half)

cross ← 0
  p   ← 0

for each y in R, ascending order

    while p < |L| & L[p] <= 2y
          do p ← p+1

cross ← cross + (|L|-p)

M ← ordinary merge of L & R

return (M, cL + cR + cross)
```

RECURRENCE RELATION:-

$$T(n) = 2T\left(\frac{n}{2}\right) + cn$$

SOLVING THE RECURRENCE

$$T(n) = 4T\left(\frac{n}{4}\right) + 2cn$$

$$T(n) = 8T\left(\frac{n}{4}\right) + 3cn$$

pattern

$$n \cdot T(1) + cn \cdot \log_2 n$$

Result:-

$$T(n) = O(n \log n)$$

$$\text{Space} = O(n)$$

CORRECTNESS:-

Every pair in the left half right half or crosses, the cross count depends only on the contents of the halves. As y increases 2y never decreases

So P only moves forward, and L[p..] is exactly the set of left elements greater than 2y.

EX #03 BANK CARDS

ALGORITHM :-

If a class holds more than $(\frac{n}{2})$ of the cards, it holds more than half of atleast one-half. So we solve both halves recursively. then verify each of the atmost 2 candidates against all cards with the tester.

PSEUDOCODE :-

```
algo Majority(Cards)
    if |cards| == 1
        then return that card
```

c1 ← Majority (first half)
c2 ← Majority (Second half)

for c in (c1, c2)

count ← [1 + no# of other cards
x Tester (c, x) = true]

if (2 * count) > |cards|
then return c

return none.

RECURRENCE RELATION:-

$$T(n) \leq 2T\left(\frac{n}{2}\right) + 2(n-1)$$

$$\therefore T(1) = 0$$

SOLVING THE RECURRENCE :-

$(n = 2^k)$

Level i $(i = 0 \ldots k-1)$ has 2^i calls, each costing $2(n/2^i - 1)$

~~$2\left(\frac{n}{2^i} - 1\right)$~~

so the level costs $2(n-2i)$

Summing over the k levels

$$T(n) = 2kn - 2(2^k - 1)$$

$$= 2kn - 2(n-1)$$

FINAL RESULT :-

$$T(n) = 2n \cdot \log_2 n - 2n + 2$$

$$= O(n \log n) \text{ tester calls}$$

Brute force needs upto
$n(n-1)$ calls &
~~Space~~

$$\text{Space} = O(n)$$

CORRECTNESS:-

By the fact above, & the ~~devide~~ mathematical rule that an overall majority element must also be a majority in atleast one of its divided

halves. Through induction this guarantees that if a true majority exists, it will survive the recursive splits & emerge as a candidate to be checked. The final linear verification step strictly imposes/enforces the $> n/2$ threshold ensuring exact counting & preventing false positives. in edge cases where an element makes up exactly half of the dataset.